

TensorFlow.js：使用 Firebase 託管服務，大規模部署及託管機器學習模型

來源：<https://codelabs.developers.google.com/codelabs/tensorflowjs-firebase-hosting-model?hl=zh-tw>

步驟數：7

在本程式碼研究室中，您將瞭解如何使用 Firebase 基礎架構部署機器學習模型，以便透過 TensorFlow.js 在網站上使用和使用模型

目錄

1. [簡介](#)
2. [什麼是 Firebase 託管？](#)
3. [預測房價的簡單機器學習模型](#)
4. [設定 Firebase](#)
5. [建立 TensorFlow.js 網頁](#)
6. [部署模型和網站](#)
7. [恭喜](#)

1. 簡介

您已使用 TensorFlow.js 建立自訂機器學習模型，但現在需要將模型託管在某個位置，以便在所選網站上使用。有很多方法可以做到這一點，但今天我們將瞭解如何輕鬆使用 [Firebase 託管](#)，這項服務還能提供版本管理、透過安全連線提供模型等額外好處。

建構項目

在本程式碼實驗室中，您將建立完整的端對端系統，能夠託管及執行自訂儲存的 TensorFlow.js 模型，以及 HTML、CSS 和 JavaScript 等相關資產。我們會建立非常簡單的輕量模型，根據輸入值預測數值輸出值 (例如根據房屋面積預測價格)，並透過 Firebase 託管服務託管模型，以便大規模使用。

課程內容

- 如何以正確格式儲存自訂 TensorFlow.js 模型
- 如何設定 Firebase 帳戶以進行託管
- 如何將資產部署至 Firebase 託管
- 如何部署新版模型。

請注意：本程式碼研究室的重點在於如何取得自訂訓練模型並託管以供部署，而非如何製作完美的模型架構，因此我們會透過簡單的範例，快速完成機器學習模型的建立。無論您最後建立哪種模型，這些原則都適用。

與我們分享你的作品

如果您使用這個堆疊打造出有趣的成果，請告訴我們！我們很期待看到你的作品。

在社群媒體上使用 [#MadeWithTFJS](#) 主題標記標記我們，就有機會讓你的專案登上 [TensorFlow 網誌](#)，甚至是日後的活動 (例如[展演秀](#))。

2. 什麼是 Firebase 託管？

Firebase 託管 可為網頁應用程式、靜態 / 動態內容和微服務提供快速又安全的實際執行環境等級代管服務

只要一個指令，即可快速部署網路應用程式，並將內容提供給全球 CDN (內容傳遞聯播網)，確保內容幾乎隨時隨地都能以低延遲時間提供。您也可以將 Firebase 託管與 Firebase [Cloud Functions](#) 或 [Cloud Run](#) 配對，建構及代管微服務，但這超出本程式碼研究室的範圍。

Firebase 託管的主要功能

- 透過安全連線提供內容 - 現代網路安全無虞。網站通常必須透過安全環境傳送，才能[存取用戶端的感應器](#)。Firebase 代管內建零設定 SSL，因此可以持續安全遞送所有託管檔案的內容。
- 託管靜態和動態內容，以及微服務，並支援驗證，因此只有登入的使用者可以載入 / 查看這些檔案 (如有需要)。
- 快速傳遞內容：您上傳的每個檔案都會以快取形式儲存在全球 CDN 邊際伺服器的固態硬碟 (SSD)。無論使用者位在哪裡，都能快速傳遞內容。
- 只要一個指令就能部署新版本：使用 Firebase 指令列介面，即可在幾秒內啟動並執行應用程式。
- 按一下即可復原 - 快速完成部署固然不錯，但如果能撤銷錯誤會更好。Firebase 託管提供完整的版本設定與版本管理功能，按一下即可執行復原程序。

無論您要部署簡單的應用程式到達網頁，還是複雜的[漸進式網頁應用程式 \(PWA\)](#)，Hosting 都能提供基礎架構、功能和工具，協助您部署及管理網站和應用程式。

根據預設，每個 Firebase 專案在 [web.app](#) 和 [firebaseapp.com](#) 網域上都有免費子網域。這兩個網站提供相同的已部署內容和設定。您也可以視需要將自有網域名稱連結至 Firebase 代管網站。

導入步驟

- 設定專案
- 安裝及設定 Firebase 指令列介面
- 部署網站
- 連結至 [Firebase 網頁應用程式](#)，進行[效能監控](#) (選用)

但在此之前，我們需要部署機器學習模型和網頁應用程式。我們來製作一個吧！

3. 預測房價的簡單機器學習模型

在本練習中，我們將建立非常簡單的 ML 模型，用於預測數值。我們將嘗試使用機器學習技術，根據虛構房屋的平方英尺大小預測其價值，僅供說明用途。事實上，在這項示範中，我們只會將房屋的平方英尺乘以 1000，藉此取得訓練資料的預測值，但機器學習需要自行學習這項資訊。

在實際情況中，您會選擇使用真實世界的資料，這些資料可能具有更複雜的關係 (例如，估算較小房屋的價值時，可能只有房屋大小的 500 倍，但超過特定門檻後，會逐漸變成 1000 倍等)，您可能需要更進階的模型，才能瞭解預測這些值的最佳方式。

我們今天建立的模型 (線性迴歸) 可用於預測許多其他事物，前提是提供足夠的實際資料，而且對於上述假設用途來說，這個模型很容易上手。不過，我們今天的重點是瞭解如何儲存及部署模型，而不是針對特定用途設計及最佳化模型。現在就開始吧！

訓練和測試資料

所有機器學習模型都會先取得一些範例訓練資料，用來教導模型預測未來值。通常這類資料會來自資料庫、檔案資料夾、CSV 等，但這裡我們會直接在 JavaScript 中將 20 個範例硬式編碼為陣列，如下所示。建議您在目前滿意的環境中複製這段程式碼，例如 [Glitch.com](https://glitch.com)，或是在本機使用自己的文字編輯器 (如果可以在 localhost 上執行伺服器)。

model.js

```
// House square footage.
const data = [800, 850, 900, 950, 980, 1000, 1050, 1075, 1100, 1150, 1200, 1250, 1300, 1400, 1500, 1600, 1700, 1800, 1900, 2000];

// Estimated dollar cost of house for each piece of data above (1000x square footage).
const answers = [800000, 850000, 900000, 950000, 980000, 1000000, 1050000, 1075000, 1100000, 1150000, 1200000, 1250000, 1300000, 1400000, 1500000, 1600000, 1700000, 1800000, 1900000, 2000000];

// Testing data separate from training data.
const dataTest = [886, 1225, 500];
const answersTest = [886000, 1225000, 500000];
```

如您所見，每筆資料都有對應的答案值，也就是我們日後會嘗試預測的值 (您可以將這些值想像成簡單 2D 圖表上的 x 和 y 值)。

因此，如果輸入值為 800，我們希望輸出 \$800,000 美元的預估答案。如果是 900，我們會輸出 \$900,000，以此類推。基本上就是將該數字乘以 1000。不過，機器學習模型並不知道 $1000 * N$ 這個簡單的關係，必須從我們提供的這些範例自行學習。

請注意，我們也有一些與訓練資料完全不同的測試資料。這樣我們就能評估訓練好的模型，瞭解模型處理未曾接收過資料時的成效。

我們將使用下列 HTML 載入這個指令碼和 TensorFlow.js 程式庫：

train.html

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <title>Training Model</title>
    <meta charset="utf-8">
  </head>
  <body>
    <!-- Import TensorFlow.js library -->
    <script src="https://cdn.jsdelivr.net/npm/@tensorflow/tfjs/dist/tf.min.js"
type="text/javascript"></script>
    <!-- Import our JS code to train the model -->
    <script src="/model.js" defer></script>
  </body>
</html>
```

訓練模型

接著，在檔案結尾的現有 JS 程式碼中加入下列程式碼，即可訓練模型。

我們已為好奇的您新增註解，但如前所述，本程式碼研究室的重點是取得已儲存的模型並加以代管。如要進一步瞭解模型建立作業，請參閱結尾連結的其他程式碼研究室。目前你可以複製程式碼並貼到專案中。

model.js

```

// Create Tensor representations of our vanilla JS arrays above
// so can be used to train our model.
const trainTensors = {
  data: tf.tensor2d(data, [data.length, 1]),
  answer: tf.tensor2d(answers, [answers.length, 1])
};

const testTensors = {
  data: tf.tensor2d(dataTest, [dataTest.length, 1]),
  answer: tf.tensor2d(answersTest, [answersTest.length, 1])
};

// Now actually create and define model architecture.
const model = tf.sequential();

// We will use one dense layer with 1 neuron and an input of
// a single value.
model.add(tf.layers.dense({inputShape: [1], units: 1}));

// Choose a learning rate that is suitable for the data we are using.
const LEARNING_RATE = 0.0001;

train();

async function train() {
  // Compile the model with the defined learning rate and specify
  // our loss function to use.
  model.compile({
    optimizer: tf.train.sgd(LEARNING_RATE),
    loss: 'meanAbsoluteError'
  });

  // Finally do the training itself over 500 iterations of the data.
  // As we have so little training data we use batch size of 1.
  // We also set for the data to be shuffled each time we try
  // and learn from it.
  let results = await model.fit(trainTensors.data, trainTensors.answer, {epochs: 500,
    batchSize: 1, shuffle: true});

  // Once trained we can evaluate the model.
  evaluate();
}

async function evaluate(stuff) {
  // Predict answer for a single piece of data.
  model.predict(tf.tensor2d([[768]])).print();
}

```

使用上述程式碼，我們已訓練出能夠根據輸入值預測輸出值的模型。執行上述程式碼後，輸入值 768 的預測結果為 768,073，並會列印到瀏覽器的開發人員控制台 (如果尚未開啟，請按 F12 開啟)。由於我們提供的範例比輸入內容高出

1000 倍，因此這項房屋價格估算結果相當準確。注意：預測值可能略有不同，這是正常現象。

如果我們對這項效能感到滿意，現在只要將這個模型儲存到磁碟，就能上傳至 Firebase 託管！

儲存模型

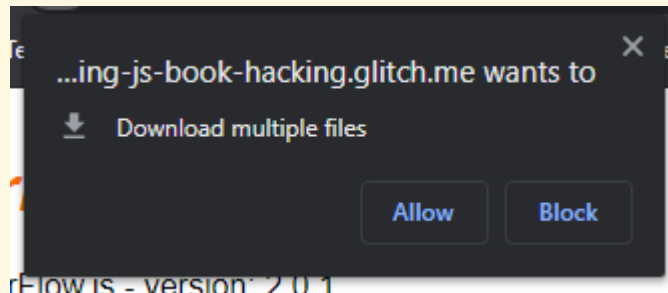
在上述評估函式 (model.predict 後方) 結尾新增下列程式碼，即可在訓練完成後直接從網頁瀏覽器匯出產生的模型，並儲存至磁碟，這樣我們就能在其他地方代管模型，日後使用時不必每次載入網頁都重新訓練。

model.js

```
await model.save('downloads://my-model');
```

現在前往 train.html 並執行該頁面，系統應該會訓練模型 (可能需要幾秒鐘)，然後在完成後提示下載訓練完成的模型。

注意：如果您是第一次嘗試在網路瀏覽器中儲存檔案，系統可能會要求您授予下載多個檔案的權限：



下載檔案前，請先點選這個彈出式視窗中的「允許」，因為系統會要求儲存多個檔案。TensorFlow.js 儲存函式會產生幾個檔案：

- my-model.json
- my-model.weights.bin (如果模型檔案大小較大，可能會有不只一個這類檔案，但以我們的簡單範例來說，只會有一個)。

如果不允許這項權限，瀏覽器可能會封鎖多個檔案的儲存作業。

步驟 4 · 約 15 分鐘

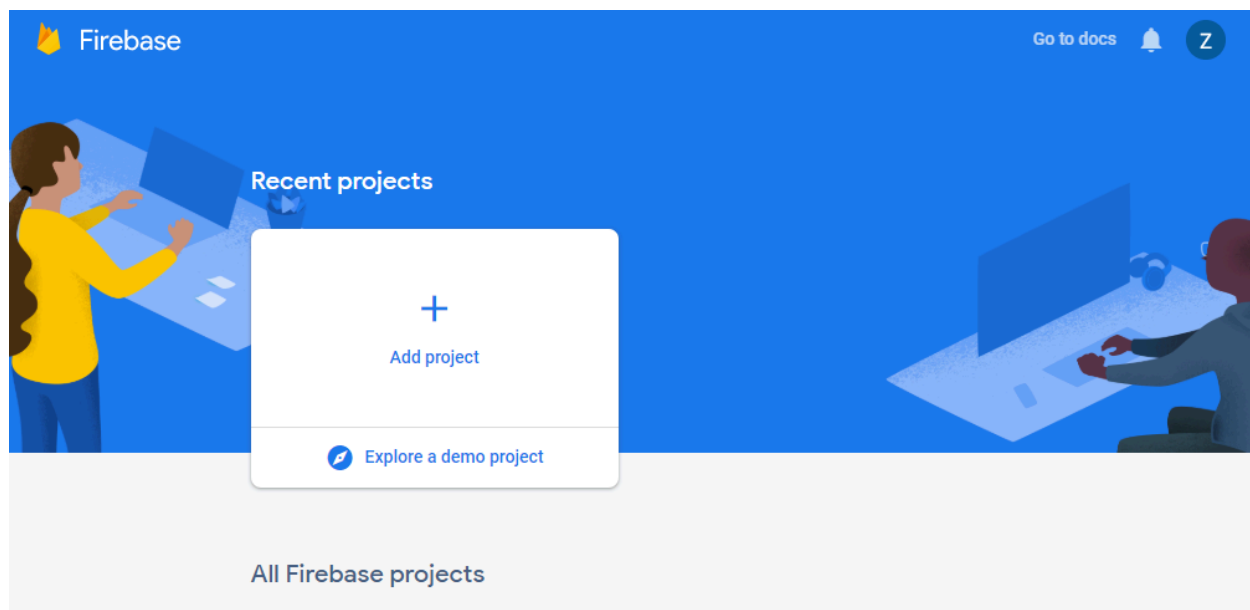
4. 設定 Firebase

登入 Firebase 並建立專案

如果您是 Firebase 新手，只要使用 Google 帳戶即可輕鬆註冊。只要前往 <https://firebase.google.com/>，然後使用您想使用的 Google 帳戶登入即可。系統將您重新導向至首頁後，請按一下頁面右上方的「前往主控台」：



重新導向至控制台後，您應該會看到類似下方的到達網頁：



只要按一下「新增專案」，即可建立新的 Firebase 專案，為專案命名專屬名稱，接受條款，然後按一下「繼續」。

接著，系統會詢問您是否要在專案中加入 Analytics。如要存取這類數據分析資料，請啟用這個選項，然後按一下「繼續」，如下所示：

Google Analytics for your Firebase project

Google Analytics is a free and unlimited analytics solution that enables targeting, reporting, and more in Firebase Crashlytics, Cloud Messaging, In-App Messaging, Remote Config, A/B Testing, Predictions, and Cloud Functions.

Google Analytics enables:

 A/B testing 

 User segmentation & targeting across
Firebase products 

 Predicting user behavior 

 Crash-free users 

 Event-based Cloud Functions triggers 

 Free unlimited reporting 

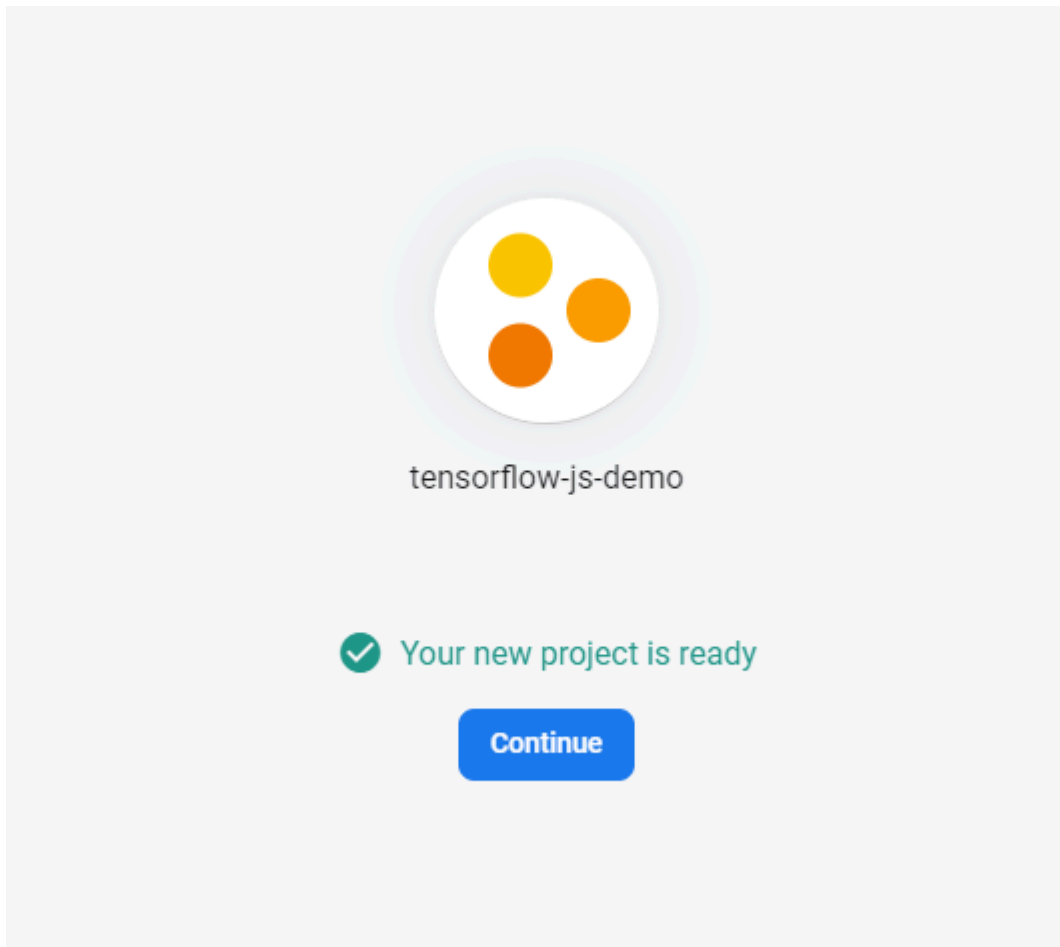
☒ Enable Google Analytics for this project
Recommended

[Previous](#)

[Continue](#)

注意：系統可能會在下一個頁面要求您接受 Analytics 條款，請視需要閱讀並接受，然後繼續操作。

如果一切順利，您應該會看到如下所示的專案就緒頁面：



太厲害了！我們有一個專案。按一下「繼續」，即可前往新建立專案的控制台。請保留這個頁面，稍後會用到，但現在必須安裝一些工具。

安裝及連線 CLI

Firebase 提供 Node NPM 套件，您可透過指令列介面 (CLI) 安裝及使用，輕鬆將本機檔案和資料夾部署至 Firebase 託管。在本教學課程中，我們將使用 Linux 環境，但如果您使用 Windows 或 Mac，可以[按照這裡的說明](#)在裝置上設定 CLI 工具。

注意：無論使用哪種作業系統，都必須使用 **Node v8.13.0** 以上版本。如果您先前已安裝 Node，可能需要升級，才能正確執行 firebase-tools 安裝作業。

不過，在 Linux 上，如果尚未安裝 NPM 和 Node.js，我們會先安裝 (如果使用其他環境，請按照 [這些操作說明](#) 操作)，方法是在終端機視窗中執行下列 3 個指令：

指令列終端機：

```
sudo apt update
```

指令列終端機：

```
sudo apt install nodejs
```

指令列終端機：

```
sudo apt install npm
```

安裝 Node.js 和 NPM 後，您只需在終端機視窗中執行下列指令，即可安裝 Firebase 指令列工具：

指令列終端機：

```
sudo npm install -g firebase-tools
```

太好了！現在我們已準備好將 Firebase 專案連結至系統，以便將檔案推送至專案等。

登入 Firebase

執行下列指令，使用 Google 帳戶登入 Firebase：

指令列終端機：

```
firebase login
```

注意：網頁瀏覽器會在幾秒後開啟，要求您登入 Firebase 並授予 Firebase 專案存取權。如果彈出式視窗沒有自動顯示，您可能需要複製 / 點選終端機視窗中顯示的連結。開啟連結後，如果尚未登入，請登入並授予 Firebase 存取權，工具才能正常運作。

系統會要求您授予 Google Firebase 帳戶存取權，如下所示：

Sign in with Google



Firebase CLI wants to access your Google Account

z

This will allow **Firebase CLI** to:

- View and manage your data across Google Cloud Platform services

i
- 

 View and administer all your Firebase data and settings

i
- View your Cloud Platform projects

i

Make sure you trust Firebase CLI

You may be sharing sensitive info with this site or app. Learn about how Firebase CLI will handle your data by reviewing its [terms of service](#) and [privacy policies](#). You can always see or remove access in your [Google Account](#).

[Learn about the risks](#)

Cancel

Allow

English (United States) ▾

Help

Privacy

Terms

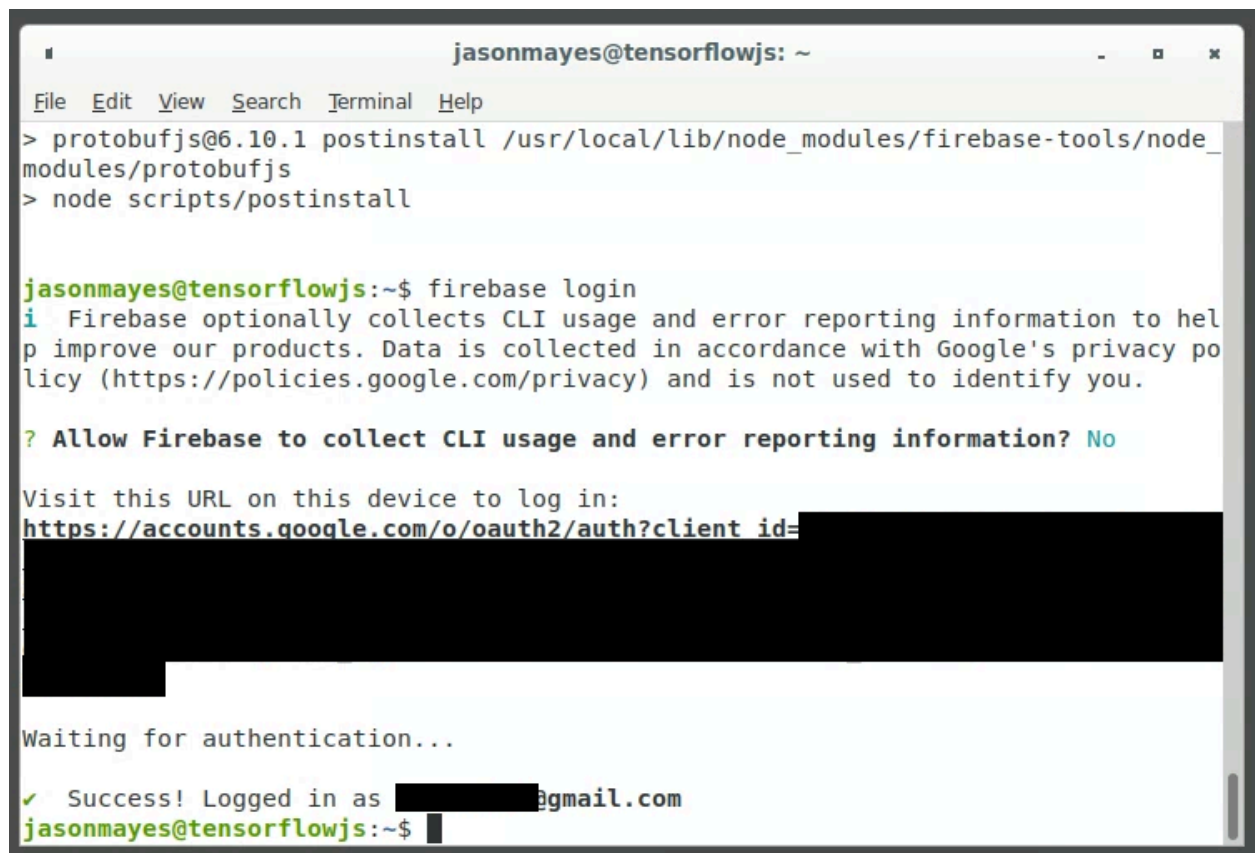
允許後，您應該就能看到指令列工具已成功連結至 Firebase 帳戶：

Woohoo!

Firebase CLI Login Successful

You are logged in to the Firebase Command-Line interface. You can immediately close this window and continue using the CLI.

關閉視窗，然後返回先前輸入指令的命令列終端機，現在應該可以接受新指令，如圖所示 (我們已在螢幕截圖中隱藏所有私人資訊)：

A terminal window titled 'jasonmayes@tensorflowjs: ~' with a menu bar (File, Edit, View, Search, Terminal, Help). The terminal shows the execution of 'firebase login'. It displays a message about data collection, asks for permission to collect CLI usage (answered 'No'), provides a URL for login, and shows the successful login for 'jasonmayes@gmail.com'.

```
jasonmayes@tensorflowjs: ~  
File Edit View Search Terminal Help  
> protobufjs@6.10.1 postinstall /usr/local/lib/node_modules/firebase-tools/node_modules/protobufjs  
> node scripts/postinstall  
  
jasonmayes@tensorflowjs:~$ firebase login  
i Firebase optionally collects CLI usage and error reporting information to help improve our products. Data is collected in accordance with Google's privacy policy (https://policies.google.com/privacy) and is not used to identify you.  
  
? Allow Firebase to collect CLI usage and error reporting information? No  
  
Visit this URL on this device to log in:  
https://accounts.google.com/o/oauth2/auth?client_id=[REDACTED]  
[REDACTED]  
  
Waiting for authentication...  
  
✓ Success! Logged in as [REDACTED]@gmail.com  
jasonmayes@tensorflowjs:~$
```

恭喜！現在可以從本機電腦將檔案推送至建立的專案。

初始化專案，以便部署至 Firebase 託管

如要將本機資料夾連結至 Firebase 專案，請從本機專案目錄的根目錄 (您要在部署時上傳檔案的資料夾) 執行下列指令。

指令列終端機：

```
firebase init
```

執行這項指令後，只要按照終端機中的指示完成設定即可，如下所示：

```
jasonmayes@tensorflowjs:~/Desktop/fbtest$ firebase init
```

```
#####  #####  #####  #####  #####  #####  #####  #####  
##      ##  ##      ##  ##      ##      ##  ##      ##  
#####  ##  #####  #####  #####  #####  #####  #####  
##      ##  ##      ##  ##      ##      ##  ##      ##  
##      #####  #####  #####  ##      ##  #####  #####
```

You're about to initialize a Firebase project in this directory:

```
/usr/local/google/home/jasonmayes/Desktop/fbtest
```

? Which Firebase CLI features do you want to set up for this folder? Press Space to select features, then Enter to confirm your choices. (Press <space> to select, <a> to toggle all, <i> to invert selection)

- ☐ Database: Deploy Firebase Realtime Database Rules
- ☐ Firestore: Deploy rules and create indexes for Firestore
- ☐ Functions: Configure and deploy Cloud Functions
- ☒ Hosting: Configure and deploy Firebase Hosting sites
- ☐ Storage: Deploy Cloud Storage security rules
- ☐ Emulators: Set up local emulators for Firebase features

在這裡，我們只要使用鍵盤上的向下鍵選取 **Hosting**，然後按下空格鍵選取，再按 **Enter** 鍵確認即可。

現在我們可以選取先前建立的現有專案：

=== Project Setup

First, let's associate this project directory with a Firebase project. You can create multiple project aliases by running **firebase use --add**, but for now we'll just set up a default project.

? Please select an option:

> Use an existing project

Create a new project

Add Firebase to an existing Google Cloud Platform project

Don't set up a default project

按下「use an existing project」的 **Enter** 鍵，然後使用向下鍵選取，如下所示：

=== Project Setup

First, let's associate this project directory with a Firebase project. You can create multiple project aliases by running **firebase use --add**, but for now we'll just set up a default project.

? Please select an option: Use an existing project

? Select a default Firebase project for this directory:

> tensorflow-js-demo (tensorflow-js-demo)

最後按下 **Enter** 鍵即可使用，然後接受彈出最終畫面上的預設值，並說「No」將其設定為單頁應用程式：

=== Hosting Setup

Your **public** directory is the folder (relative to your project directory) that will contain Hosting assets to be uploaded with **firebase deploy**. If you have a build process for your assets, use your build's output directory.

```
? What do you want to use as your public directory? public
? Configure as a single-page app (rewrite all urls to /index.html)? No
✓ Wrote public/404.html
✓ Wrote public/index.html

i Writing configuration info to firebase.json...
i Writing project information to .firebaserc...
i Writing gitignore file to .gitignore...

✓ Firebase initialization complete!
jasonmayes@tensorflowjs:~/Desktop/fbtest$
```

這樣一來，您就能代管多個 HTML 網頁。

初始化完成後，您會發現 `firebase.json` 檔案和「public」資料夾已在我們執行上述指令的目錄中建立。



public



firebase.json

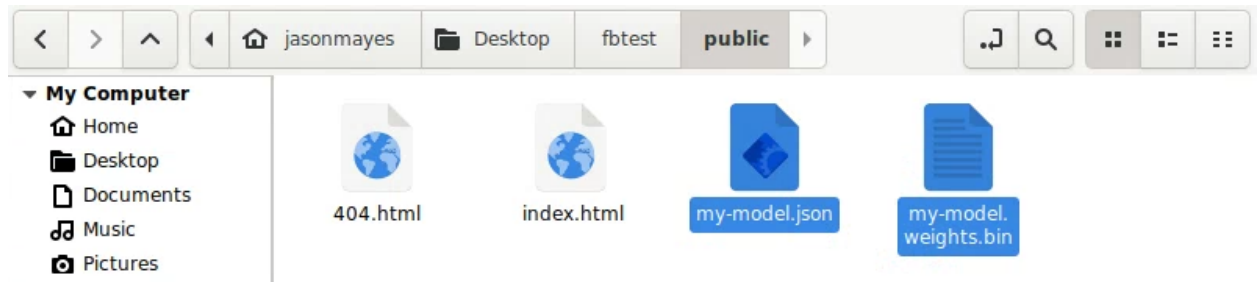
現在只要將要部署的檔案移至我們建立的公開資料夾，即可部署！開始行動吧。

步驟 5 · 約 15 分鐘

5. 建立 TensorFlow.js 網頁

載入已儲存的模型

首先，請務必將稍早在程式碼研究室中儲存的機器學習模型，複製到我們剛才使用 Firebase 建立的公開資料夾。只要將儲存的檔案拖曳到這個資料夾即可，如下所示：



您也會發現 Firebase 為我們建立了 index.html 和 404.html 檔案。請使用電腦上慣用的文字編輯器編輯 index.html，加入自訂程式碼，如下所示：

index.html

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <title>Hello World - TensorFlow.js</title>
    <meta charset="utf-8">
    <meta http-equiv="X-UA-Compatible" content="IE=edge">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <!-- Import the webpage's stylesheet -->
    <link rel="stylesheet" href="style.css">
  </head>
  <body>
    <h1>TensorFlow.js Hello World</h1>
    <p>Check the console (Press F12) to see predictions!</p>
    <!-- Import TensorFlow.js library -->
    <script src="https://cdn.jsdelivr.net/npm/@tensorflow/tfjs/dist/tf.min.js"
type="text/javascript"></script>

    <!-- Import the page's JavaScript to do some stuff -->
    <script src="script.js" defer></script>
  </body>
</html>
```

在上述 index.html 的新程式碼中，我們指定了 CSS 樣式表，因此稍後可以視需要為網頁新增樣式，並指定 script.js 來存放使用 TensorFlow.js 儲存模型時需要編寫的程式碼。

現在就來建立這些檔案，並填入下列內容，讓檔案發揮實用功能：

style.css

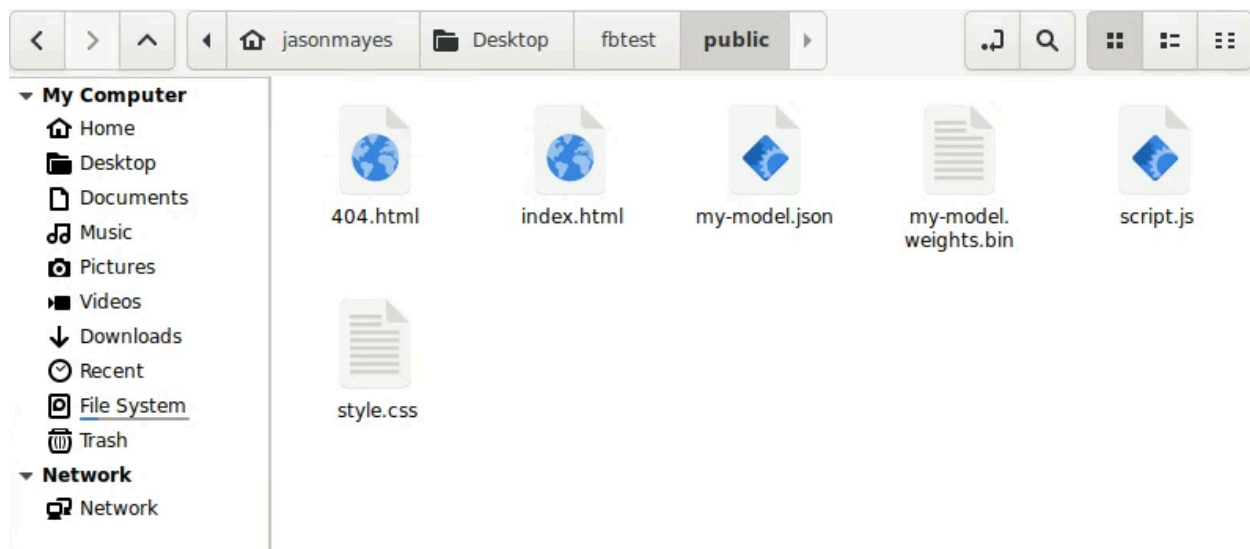
```
/** Leave blank for now **/
```

script.js

```
// Load our saved model from current directory (which will be
// hosted via Firebase Hosting)
async function predict() {
  // Relative URL provided for my-model.json.
  const model = await tf.loadLayersModel('my-model.json');
  // Once model is loaded, let's try using it to make a prediction!
  // Print to developer console for now.
  model.predict(tf.tensor2d([[1337]])).print();
}

predict();
```

如果正確完成上述步驟，您應該會在建立的 public 資料夾中看到下列已編輯的檔案：



現在只要部署檔案，就能檢查是否正常運作！

步驟 6 · 約 10 分鐘

6. 部署模型和網站

即將開始直播

返回在本機電腦的 Firebase 專案資料夾中開啟的終端機視窗 (這個資料夾包含上述的「public」資料夾和 Firebase 初始化檔案)。

只要輸入下列指令，即可部署公用資料夾檔案：

指令列終端機：

```
firebase deploy
```

讓終端機指令完成，您應該會成功發布版本，並取得可使用的網址：

```
jasonmayes@tensorflowjs:~/Desktop/fbtest$ firebase deploy

=== Deploying to 'tensorflow-js-demo'...

i  deploying hosting
i  hosting[tensorflow-js-demo]: beginning deploy...
i  hosting[tensorflow-js-demo]: found 6 files in public
✓  hosting[tensorflow-js-demo]: file upload complete
i  hosting[tensorflow-js-demo]: finalizing version...
✓  hosting[tensorflow-js-demo]: version finalized
i  hosting[tensorflow-js-demo]: releasing new version...
✓  hosting[tensorflow-js-demo]: release complete

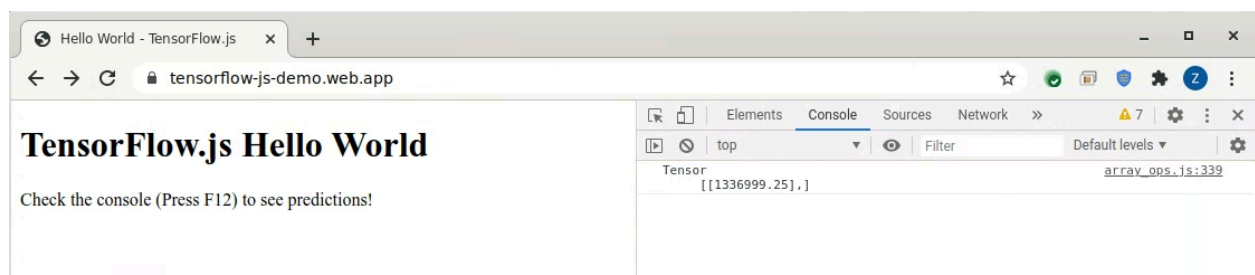
✓  Deploy complete!

Project Console: https://console.firebase.google.com/project/tensorflow-js-demo/overview
Hosting URL: https://tensorflow-js-demo.web.app
jasonmayes@tensorflowjs:~/Desktop/fbtest$
```

在上述範例中，您可以看到部署作業的最終網址為：

<https://tensorflow-js-demo.web.app> (但您的網址會是您建立的專案名稱)。

在網頁瀏覽器中開啟這個網址，確認網址是否正常運作。如果成功，您開啟的頁面開發人員控制台應該會列印類似下列內容 (按下 F12 開啟開發人員控制台)。



如您所見，頁面會在已部署的網域上載入，且我們能正確看到模型的預測結果，也就是 1337 平方英尺的 \$1,336,999.25，這確實是非常好的估計值，因為我們預期這會是平方英尺的 1000 倍。當然，如果我們建立一個良好的使用者介面來呼叫模型，就可以盡可能做出預測，而且這一切都會完全在 JavaScript 中執行，確保查詢的隱私和安全性。

注意：您只需要在載入網頁時載入模型一次，之後就能隨意使用不同的輸入內容呼叫 `model.predict`。

模型部署及代管完成後，您就能與全世界分享網站，讓使用者在自己的電腦上使用應用程式。您可能想新增更完善的使用者介面，讓應用程式看起來更美觀，但這已超出本教學課程的範圍。您可透過機器學習技術，以這種方式託管無數個網頁應用程式，這些應用程式只需按一下即可運作，完全不需要安裝。我們建議您思考其他情況，瞭解瀏覽器內機器學習模型可帶來的效益。

監控使用情形

除了可新增至網站程式碼的 Google Analytics 外，Firebase 也提供專案的版本管理和使用情況統計資料 (透過[控制台](#))。部署完成後，您會看到類似下方的內容，可視需要不時查看：

Hosting

tensorflow-js-demo domains

Domain	Status
tensorflow-js-demo.web.app	Default
tensorflow-js-demo.firebaseio.com	Default

tensorflow-js-demo release history

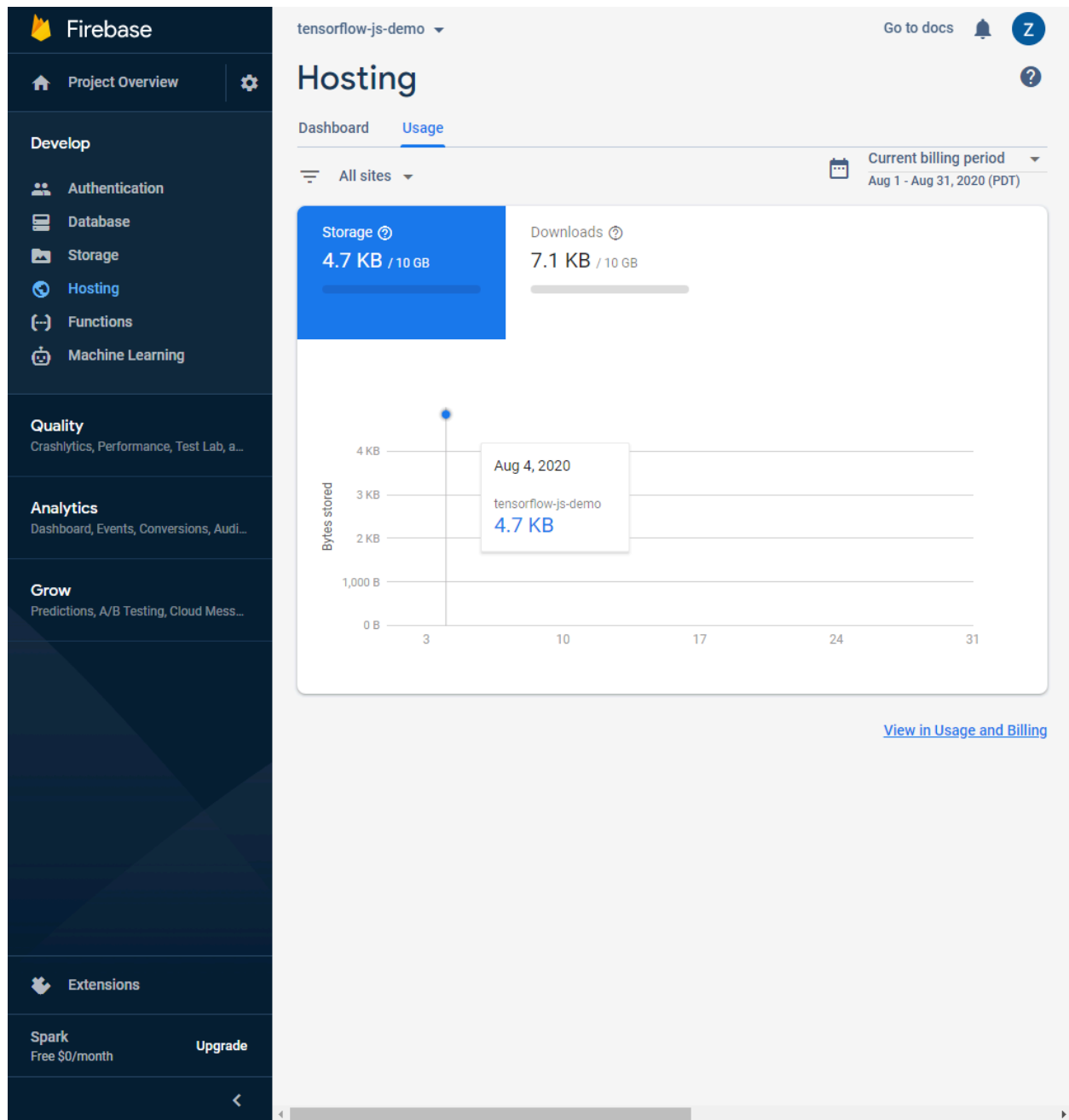
Status	Time	Deploy	Files
★ Current	Aug 4, 2020 2:03 PM	Z [REDACTED]	8
📦 Deployed	Aug 4, 2020 1:59 PM	Z [REDACTED]	8

Advanced

Firebase Hosting supports multiple sites

You can associate multiple sites with one Firebase project. Each site hosts its own unique static assets, but shares Firebase resources with the other sites in the same Firebase project.

[Add another site](#)



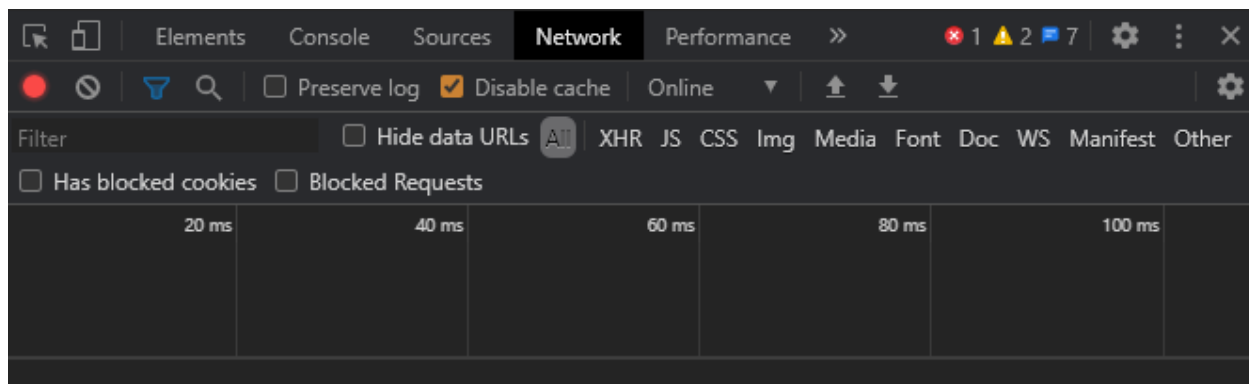
如您所見，免費方案預設提供每月 10 GB 的頻寬，供您使用代管檔案。如果網站流量較高，您可能需要新增帳單帳戶，才能在一個月內使用超過此額度的資源。如要瞭解大型專案適用的 Firebase 方案，請參閱[這篇文章](#)。不過，如果模型不大且用量不高，大多數原型製作的休閒使用者可能不會超過免費方案，因此在業務或構想成長之前，這是測試及確認是否符合需求的絕佳方式，不必急著訂閱付費方案。

步驟 7 · 約 0 分鐘

7. 恭喜

恭喜！您已完成使用 TensorFlow.js 和 Firebase 建構及部署自訂機器學習模型的第一步，現在可以與全世界分享。試想一下，您還能運用這個強大且可擴充的方法製作哪些內容。如果您願意，Firebase 會根據需求自動調整規模，因此無論是 10 位還是 10,000 位使用者想使用這項功能，都能順利運作。

如果變更任何檔案，只要像之前一樣使用 `firebase deploy` 重新部署應用程式，並務必清除瀏覽器快取，確保下次載入頁面時取得新版檔案。如果您已開啟開發人員工具，可以在測試項目時，選取這個分頁頂端附近的「停用快取」核取方塊，強制執行這項操作，方便您進行測試：



重點回顧

在本程式碼研究室中，我們：

1. 從頭定義及訓練自訂 TensorFlow.js 模型，以預測房價。
2. 已在開發機器上註冊、設定及安裝 Firebase + Firebase CLI 工具。
3. 部署並推出可運作的網站，從步驟 1 載入訓練好的模型，並在可供全球各地使用者存取的實際網頁應用程式中大規模使用。

後續步驟

您現在已具備可供使用的基礎，可以發想哪些創意，擴充這個機器學習模型部署樣板？

我們很期待您能使用自己的資料來試試這項功能。想想您居住或工作的產業或領域。您該如何根據這類資料進行訓練，以便在日後做出對自己 (或他人) 有用的預測？房地產只是其中一個例子，建議您也將這項技術應用於自身面臨的挑戰。祝你駭客松順利！

請記得使用 [#MadeWithTFJS](#) 主題標記，在社群媒體上分享你使用 TensorFlow.js 創作的內容 (按一下這個連結，查看其他人的作品)，就有機會在社群媒體上獲得曝光，甚至在日後的 TensorFlow 活動中展出！我們很期待看到您的作品，如有任何意見回饋或問題，歡迎隨時與本程式碼研究室的[作者](#)聯絡。

更多 TensorFlow.js 程式碼研究室，深入瞭解相關主題

- [使用 TensorFlow.js 從頭開始編寫類神經網路](#)
- [在 TensorFlow.js 中使用遷移學習技術辨識音訊](#)
- [使用 TensorFlow.js 中的遷移學習功能進行自訂圖片分類](#)

值得一訪的網站

- [TensorFlow.js 官方網站](#)
- [TensorFlow.js 預先建構模型](#)

- [TensorFlow.js API](#)
 - [TensorFlow.js 示範](#)
-